DEMO ENVIRONMENT

Features Shown off:

- PDF upload of simple contract
- Read PDF and hashes necessary info
- Creates smart contract
- Confirms asset transfer
- Verifies all users
- Stores information/viewable data
- Clean demo UI
- Show off the data processing that can be achieved by this system (Structured data viewed in the dashboard and the data export feature)
- Cap tables
- Automatic Waterfall Calculations derived from the LPA and ownership structure

What we are intending to prove (What is the value of using a blockchain based infrastructure):

- Utilising the Kadra infrastructure data is more integrable
    - Tamper-resistant, time-stamped, independently verifiable records
- Proving there are fewer failure modes than the current system
    - Showing system could have reduced attack surface, improved integrity, and provable auditability
    - Most breaches happen at the handoff points from one system to another. Kadra removes the handoffs
    - Even if Kadra's application layer were compromised, the ledger history cannot be rewritten
- Current systems rely on trust in people and processes. Kadra relies on verifiable systems and minimized trust.
- The potential audit packages that can be generated and exported from the Kadra system
- Automatic settlement logic
- Efficiency
- Overall simplicity, a more streamlined process, cutting out redundancy
- Time savings
- Cost reduced

Demo Components

The demo is composed of legal authority, identity permissioning, settlement validation, deterministic enforcement, data generation, data structuring and native auditability. All shown through a single fund dashboard.

- Legal and Governance Components(Replica Documents)
    - Master LPA
        - Single uploaded PDF
        - Defining:
            - Fund structure (ownership percentages and distributions)

© 2026 Markus Lam. All Rights Reserved.

            - Ownership
            - Transfer restrictions (LP-only transfer)
            - Capital committed
            - Legal sentence defining that the asset is legally represented on-chain (whatever preq exist in the signed deal that need to be enforced???)
        - PDF stored off-chain
        - Document content stored as:
            - Immutable hash
            - Time-stamped reference
    - Subscription Records
        - Investor Identity
            - Who is subscribing
        - Allocation amount
        - Reference to governing document
        - Wallet address
        - Status
            - Pending: Intent of ownership, but not settled
            - Settled: Payment confirmed, ownership finalized
        - TImestamps
            - When the subscription was created
            - When it was settled
        - References
            - Master LPA hash
            - Investor Identity
            - Allocation %
        - States
            - Pending
            - Settled
        - Created before live demo
- Identity and Wallet Components
    - Investor Registry
        - Pre-approved and pre-structured LPs
        - Fund-specific eligibility marker
    - Wallet Mapping
        - Each investor mapped to one wallet address
    - Wallets are:
        - Visible
        - Deterministic: Each investor is mapped to exactly one wallet address
        - Non-cutodial: Investors control their own wallets
- Settlement Components
    - Mock Payment Confirmation PDF
        - Verified against:
            - Subscription amount

© 2026 Markus Lam. All Rights Reserved.

            - Investor identity
            - Document reference
    - Settlement Validator
        - Confirming:
            - Amount (does it match with the uploaded LPA)
            - Investor Match
            - Document linkage: Ensures payment settles payment settles the specific subscription, not a different one. Have a payment reference and a matching subscription ID.
            - Validator confirms: Payment linked to correct subscription
        - Flipping the state from pending to settled
- Smart Contract Components
    - ERC-3643 Security Token
        - Deployed after settlement
        - Encodes:
            - Transfer restrictions
            - Identity checks
        - Represents Ownership interest
    - Identity/Compliance Module
        - Linked to investor registry
        - Checks eligibility on every transfer(blocks illegal transfers deterministically)
- Data and Ledger Components
    - On-Chain Ledger
        - Records:
            - Token minting
            - Transfers
        - Immutable
        - Time-stamped
    - Off-Chain Event Log (Compiled via pulling mechanism, essentially takes the info from the blockchain and makes it visible and understandable to the user)
        - Mirrors:
            - Document uploads
            - Settlement events
        - Linked via hashes
    - Capital Account Module (Off-chain structured layer)
        - For each LP:
            - Initial capital contribution
            - Date contributed
            - Ownership %
            - Capital returned
            - Accrued Preferred Return
            - Residual allocation
            - Ending capital balance

© 2026 Markus Lam. All Rights Reserved.

        - These are updated automatically when:
            - Settlement occurs
            - Transfers occur
            - Distribution simulation occurs
- Waterfall Configuration Module
    - Visible in UI:
        - Calculation basis (IRR vs multiple)
        - Distribution frequency
        - Catch-up: yes/no
    - Distribution Calculation Flow:
        - Pull settled ownership ledger
        - Pull capital accounts
        - Apply waterfall tiers sequentially
        - Update capital accounts
        - Generate per-LP breakdown
        - Append event log entry
- Fund Dashboard
    - Fund Overview
        - Fund name
        - Allocation %
        - LP count
        - Cap table
    - Ownership Timeline
    - Transaction history
- Rule Enforcement Components
    - Illegal Transfer Attempt
        - Non-LP wallet
        - Rejected transaction
        - Visible error
    - Legal Transfer Attempt
        - LP to LP
        - Successful transfer
        - Instant dashboard updates
- Audit & Export Components
    - Structured Data Views (viewed in dashboard)
        - Ownership table
        - Event history
        - Document references
    - Export Engine
        - Cap table
        - On-chain references included

Pre-Demo Setup

© 2026 Markus Lam. All Rights Reserved.

We should arrive with this setup

What is preconfigured:

- Fund entity created
    - Name: Kadra Demo Fund I
    - Legal form: LP (or LLC taxed as partnership)
    - Jurisdiction metadata stored (stored for reference only)
    - State: No ownership issued, no capital settled
- Legal and compliance documents
    - Docs ready to upload
    - Fund-specific eligibility criteria
- Waterfall rules pre-configured but not yet executed
    - Derived from Master LPA
    - Locked and versioned
- Identity layer initialized
    - The 5 LP identities pre-approved
        - Identity verified
        - Wallets linked
        - Marked on and off chain as eligible LPs
        - Wallets “stamped with Fund-specific “LP eligibility” marker
    - Transfer rules pre-defined
        - Transfers allowed only:
            - Between wallets that hold the LP eligibility marker
- ERC-3643 (permissioned security token) template
    - Not deployed yet
    - Parameters ready:
        - Transfer restriction mechanism prepared
        - Compliance hooks
        - Identity registry linkage
- Fund Dashboard

Demo Process Procedure

1. Acknowledge the fragmented analog system of siloed ledgers and manual inputs and enforcement.

The executives see the Kadra fund dashboard

- Fund name: Kadra Demo Fund I
- Ownership allocated: 0%
- Active LPs: 0
- Cap table: Empty
- The 5 LP identities that show pending

Fund, rules, Investors all exist but ownership does not

© 2026 Markus Lam. All Rights Reserved.

2. Convert executed legal intent into structured, verifiable, machine-readable state without creating ownership.

Preconditions:

- The 5 investors already onboarded
    - Wallets linked
    - Marked off and on-chain as eligible LPs for this fund
- One Master LPA
    - Governs the fund
    - Defines:
        - Ownership
        - Transfer restrictions
    - Same for all LPs

Uploading the Master LPA

On the frontend we drag and drop the LPA PDF into the upload document section of the dashboard

Kadra will:

- Parses the document
- Extract governing clauses:
    - Fund identity
    - Ownership
    - Transfer restrictions
- Generates cryptographic hash (SHA-256)
- Anchors (permanently recording the cryptographic fingerprint of the document so the document's existence and contents can be independently verified):
    - Hash
    - Timestamp
    - Document reference
- Registers the LPA as the governing agreement for the fund

For each of the 5 investors:

- Create a subscription record that references:
    - The LPA uploaded
    - The investor identity (A,B,C,D,E)
    - Allocation amount (X%)
- Demonstrated as a cap table of some sort
- Registering per subscription (per investor):
    - Investor ID (is the wallet A or B)
    - Wallet address
    - Allocation percentage
    - References:
        - LPA hash
        - Subscription record identity
    - State: Pending

Through the fund dashboard it will show:

© 2026 Markus Lam. All Rights Reserved.

- Investors (5 LPs listed)
- Allocations per investor
- Status per investor: Pending
- Linked LPA hash
- Timestamp
- Cap table

Replacing email threads, PDF folders, and Spreadsheet rows
Legal reality is now structured, verifiable, and canonical. Nothing is lost, misread, or re-entered.

3. Show the system before settlement and how it doesn't prematurely create ownership. Also show the cap tables and the set up.

Dashboard updates:

- Lp list shows:
    - Investor A: 20% (pending)
    - Investor B: 20% (pending)
    - Investor C: 20% (pending)
    - Investor D: 20% (pending)
    - Investor E 20% (pending)
- Cap table displays:
    - Intended ownership
    - Labeled Pending/settled
- Each line item links to:
    - Subscription record
    - Governing LPA hash
    - Timestamped reference

This shows ownership intent.
The Canonical subscription ledger (off-chain, structured) is created.
A visible, auditable structure.

4. Recognize economic settlements in real life deterministically once off-chain settlement is confirmed.

We upload a mock bank wire confirmation for each investor matching the amount in the LPA
So Document includes:

- Amount
- Investor/Sender identity (A,B,C,D,E)

Kadra verifies (automated):

- Payment amount matching subscription
- Investor identity
- Document hash matches prior references
- No double settlement

The upload screen is seen.
So it would look like for each LP status : Verifying
Automatically checks the amount, investor, document reference.
Then status flips: Pending to Settled

© 2026 Markus Lam. All Rights Reserved.

System actions:

- Once verification succeeds:
    - Ownership state updates: intent to final ownership
- Ownership ledger finalized
- ERC-3643 contract:
    - Deployed or activated
    - Compliance rules embedded
- Tokens minted:
    - Exactly matching ownership %
    - Sent to LP's wallet
- Ledger entry:
    - Timestamped
    - Immutable
    - Linked to documents

Fund dashboard updates live

What it shows:

- Investor A: X amount 20% of the fund: settled
- Cap table:
    - The finalised owners
    - The ones still pending
- Activity log:
    - X ownership recognised, settlement confirmed
    - Timestamped

Then upload a bank wire that is less than the amount agreed in the LPA

- Statement will be scanned and ingested
- Statement is parsed
- Amount discrepancy detected automatically

Status remains: Pending, Settlement mismatch gets flagged

System message: Settlement amount does not match subscribed commitment

- No tokens minted
- No partial ownership created

Batch upload the rest of the "bank statements"

- Show how during mixed states the cap table is always correct and that no manual reconciliation is ever required
- Showing that Kadra is fund-grade infrastructure, ambiguity and back-office risk disappears

5. Frontend: Fund Dashboard (Final State)

Cap table should show all the investors at 20% and settled

Totals:

- Ownership allocated
- Active LPs: 5

Ownership detail

Click on any LP:

© 2026 Markus Lam. All Rights Reserved.

- Ownership percentage
- Wallet address
- Settlement timestamp
- Token Balance (ERC-3643)
- Governing LPA hash
- Subscription record ID
- Payment confirmation reference

Ledger view:

- On-chain contract address
- Token/LPA supply
- Transfer restrictions active
- Ownership changes
- Timestamps for every process

Audit-ready export

Viewed on the Audit interface and has the function to export:

- Final cap table
- Ownership ledger snapshot
- All document hashes(LPA, subscriptions, payments)
- Settlement timestamps
- Transaction history
- Contract address and verification link

6. Distribution logic and automatic waterfall setup

Ownership is:

- Final
- Settled
- Immutable

Show a predefined Waterfall Configuration Panel:

- Preferred return X%
- Return of capital X%
- IRR X%
- GP promote X%

Waterfall rules are:

- Configured from governing documents

Bound to:

- Ownership ledger
- Capital accounts
- Distribution events

No discretion and no human spreadsheet modeling

This waterfall is not an assumption. It is derived directly from the legal agreements and applied to the live ownership ledger.

© 2026 Markus Lam. All Rights Reserved.

7. Transfer Rule Enforcement (Security Proof)

Illegal transfer attempt

- LP A initiates a transfer to un-approved LP for this fund
    - Rejected immediately
    - No pending state
    - No partial execution
- Error shown: Transfer blocked: recipient is not an approved Limited Partner for Kadra Demo Fund I

On-chain rule demonstrated (highlight):

- Wallet “stamp check”
    - Fund-specific LP eligibility marker required
- ERC-3643 transfer hook invoked
- Falis pretransfer validation

8. Legal Transfer execution

Lp wallet #1 initiates a transfer:

- Recipient: LP Wallet #2
- Both wallets:
    - Identity verified
    - Carry the fund-specific LP eligibility marker

Transfer outcome:

- Transfer succeeds
- Executed on-chain
- No manual approvals
- No compliance review step

Immediate system updates:

- Token balance updates for both wallets
- Cap table recalculates automatically
- Ownership ledger adds a new entry
- Timestamp and transaction hash recorded

Frontend:

- Cap table refreshes instantly
- Ownership percentages reflect the transfer
- LP wallet #1: reduced position
- LP wallet #2: increased position
- History panel shows
    - Transfer event
    - Form/to wallets
    - Timestamp
    - On-chain transaction reference

After legal LP-to-LP transfer, the dashboard must show:

- Capital account moves proportionally
- Accrued pref follows ownership

© 2026 Markus Lam. All Rights Reserved.

9. Structured data, audit, and export

Showing:

- Canonical Ownership table (clean structured view)
    - Investor name
    - Wallet address
    - Ownership percentage
    - Status
    - Last updated timestamp
    - Governing LPA reference (hash)
- Event and Evidence Log
    - Each entry shows:
        - Event type
            - LPA ingestion
            - Subscription creation
            - Settlement confirmation
            - Token/digital asset issuance
            - Transfers
        - Actor (system/LP/GP)
        - Timestamp
        - Document hash or transaction hash
        - Immutable reference ID
- Document Provenance (system can prove which document a record came from, when it was ingested, and that it has not been altered since)
    - Show:
        - Original PDF reference
        - Cryptographic hash
        - Timestamp
        - On-chain anchor reference
- Version history of events
    - Waterfall Version 1.0
    - Derived from LPA Hash: XYZ
    - If rules ever change:
    - Version 1.1

Export Auditor Package (button)

Document types:

- CSV
    - Cap table
    - Ownership history
- PDF
    - Legal document hash registry
    - Settlement timeline
    - Transfer log
- On-chain proofs
    - Contract address

© 2026 Markus Lam. All Rights Reserved.

- Transaction hashes
- Block timestamps

Fewer handoffs=fewer breaches
Rules enforced before anyone can break them
Ownership is provable and auditability is native

Technical Appendix and preliminary research: recommendations not mandatory, key functions outlined

Frontend/UI Layer

- Next.js : Core framework for building the application dashboard.
- Tailwind CSS & shadcn/ui: Component libraries to enforce a clean, institutional banking aesthetic (strict avoidance of "crypto" UI).
- Ethers.js or Wagmi: Libraries to facilitate communication between the frontend and the local blockchain node.

Database and Off-Chain State Management

- Database Engine: Local PostgreSQL or SQLite.
- ORM: Prisma or Drizzle to manage the canonical off-chain subscription ledger.

Local Blockchain Environment

- Hardhat: Deployed as a persistent local node to simulate the EVM environment locally without gas fees.
- EVM Snapshots: Used to save and instantly reset the blockchain state to the exact starting point prior to each demo run.
- Seeding Scripts: Automated developer scripts to pre-populate the local database and whitelist the 5 mock investors upon initialization.

Smart Contracts and Token Standards

- Language: Solidity, utilizing standard OpenZeppelin security libraries.
- Token Standard: ERC-3643.
- Transfer Hooks: Must implement the hook to query the on-chain Identity Registry and deterministically revert any unauthorized transfer attempts.

Backend Logic and Cryptography

- File Handling: Multer (or equivalent multipart form parser) for ingesting the Master LPA and wire receipts.
- Hashing Engine: SHA-256 algorithm to generate the cryptographic anchor for the uploaded Master LPA PDF. (openAI or any other ai platform can be used aswell)

© 2026 Markus Lam. All Rights Reserved.

- Settlement Validator: A Node.js backend script that verifies the 3-way wire match and generates an ECDSA cryptographic signature, which the smart contract requires to authorize the token mint function.

© 2026 Markus Lam. All Rights Reserved.